{EPITECH}

ECORP_

< BOOTSTRAP />

ECORP

PWNTOOLS

Your goal for this bootstrap is to learn how to use **Pwntools**. Pwntools is a framework that describes itself as a grab-bag of tools to make exploitation during CTFs as painless as possible, and to make **exploits** as easy to read as possible.

If you're already familiar with this tool, here is a quick list of things you should be able to do. You can also use this list if you want to do this bootstrap with less guidance.

- ✓ Find and reformat 32/64bit adresses in little/big endian
- ✓ Breaking execution at an entry point
- ✓ Convert assembly into shellcode
- ✓ Searching for gadgets in the binary through a ROP object

Ressources

Repository : https://github.com/Gallopsled/pwntools
Tutorials : https://github.com/Gallopsled/pwntools-tutorial
Documentation : https://docs.pwntools.com/en/latest/





Ex 0 : Setup & Hello World

Install pwntools, and get it to work. Copy the following source code and compile it. Use pwntools to execute it.

Terminal

```
int main(void)
{
printf("Hello World")
printf("Here is an adress : %p", &knockknock);
}
void knockknock(void)
printf("Hello you");
```

Find a way to get the "Hello World" string from your pwntools script. Then, still in your script, isolate the knockknock pointer.

Use a command that would be the equivalent of "read until you see a ':' character"

Ex 1 : Context and Architecture

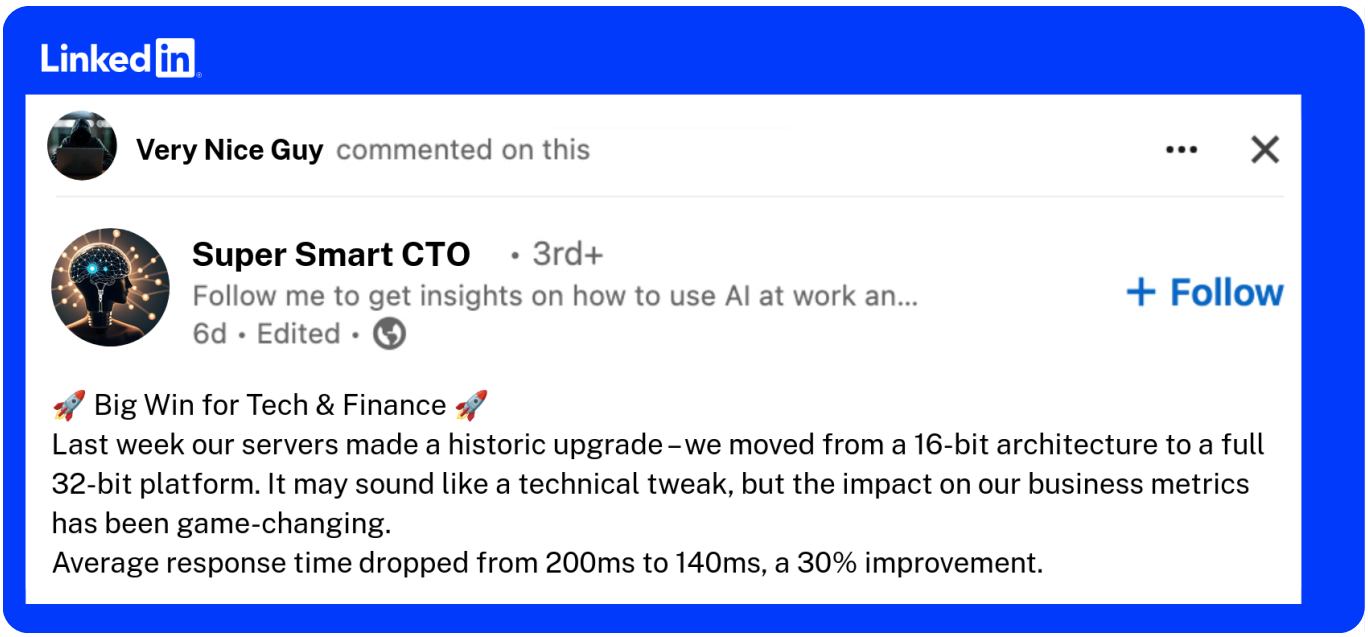
The context object is a global object which contains various settings used by pwntools.
Generally at the top of an exploit, you'll find something like:

Terminal

```
from pwn import *context.arch = 'amd64'
```

This informs pwntools that shellcode generated will be for amd64, and that the default word size is 64 bits.

The “update” method allows you to set multiple values at once. Setting these values is important, as some tools will not work correctly if you haven’t done so.



While this tactic is not used in this project (since you already know the architecture binaries were compiled on from the elf header), in other contexts, light OSINT or social engineering could give you this information. Therefore, on the flip side, be mindful about what information you leave online.

Let pwntools auto-detect the right context from the binary for this exercise.

Experiment compiling the previous source code with the **-m32 compilation flag** and look at the adress: How does it change? Can you think about or search for contexts where pwntools would not guess the right context?

Ex 2 : Shellcode

You've seen how to read code, but pwntools also allows you to **write** some. Since you're interracting with an already-compiled program, you won't write it in C, but in **assembly**. You'll then use the asm tool and/or shellcraft in order to inject your instructions into the binary.

One simple use case for writing your own piece of code is when you spot a buffer overflow. On its own, it might not do much, but if you inject your own code, you might be able to make the reversed binary's own lecture head execute your payload.

Terminal

```
#include <stdio.h>
char gets(char s);
int main(int argc, char **argv) {
char buffer[48];
gets(buffer);
return 0;
}
```



This will only work with binaries compiled with the flag "-fno-stack-protector -z execstack"



"-fno-stack-protector" will also make the process a lot easier, but this exercise could be passed without it, if you were up to the challenge.



Ex 3 : ROP and Gadgets

Injecting code is great, but the injected code is written on the **stack**. By today's standards, the stack is **not executable**, and code injections are not possible. Here is the trick to bypass this obstacle : Use the binary's own code.

The answer lies in **Gadgets**. Gadgets are subroutines that already exist in the code. They are short, perform some useful work (e.g. loads a register, move data, clear a flag) and then hand control back to whatever address is now on the stack via a return instruction.

Instead of writing a payload like in the previous exercise, you could stitch together those gadgets, using a technique called **ROP (Return-Oriented Programming)** which you then force the CPU to execute.

Ex 4 : What now?

Pwntools is a great set of tool, but it is not the only one that exists, and as all tools, it is only as good as your knowledge and technique allow. Be curious, look for other paths, ask yourselves questions, and be creative.

See you on the leaderboard!

v1

